# EE-2003 Computer Organization and Assembly Language


## LAB 04

## Register Indirect Addressing, Register + Offset Addressing

## Branching: Comparison and Conditions, Conditional Jumps

Course Code: EE-2003

Semester: Fall 2025

Instructor: Engr. Muhammad Qasim

Email: Muhammad.qasim@nu.edu.pk

Department of Computer Science, National University of Computer and Emerging Sciences FAST Peshawar

# 1. Objective

This lab is designed to familiarize you with dynamic memory access techniques and fundamental program control flow. After completing this lab, you'll be able to:

- Understand and implement **register indirect** and **register + offset** addressing modes.
- Implement decision-making in your code using **conditional jump** instructions.
- Write assembly programs that can iterate through data arrays to perform calculations.

# 2. Theoretical Background

### 2.1 Moving Beyond Fixed Addresses

So far, we've used **direct addressing** (e.g., mov ax, [num1]), where the memory address is fixed within the instruction. This is inflexible for tasks that need to process consecutive memory cells, like iterating through an array. To handle this, we need a way to use a register to hold the address of our data so it can be changed during program execution.

### 2.2 Register Indirect Addressing

In this mode, a register holds the memory address of the data we want to access. The iAPX88 architecture designates four specific registers for this purpose:

**BX, BP, SI, and DI**. When one of these registers is enclosed in brackets

[], the processor uses its content as a memory address.

- **Based Addressing:** When **BX** or **BP** is used to hold the address. The 'B' in their names stands for **Base**.
- **Indexed Addressing:** When **SI** (Source Index) or **DI** (Destination Index) is used.

By changing the value in these registers (e.g., by adding 2 to point to the next word), we can easily move through data in memory using the same instruction repeatedly.

### 2.3 Register + Offset Addressing

This mode is a powerful combination of direct and indirect addressing. It allows you to add a fixed offset (like a label) to the value in a base or index register to form the final *effective address*. This is especially useful for accessing elements in an array, where the label can point to the start of the array and the register can hold the index.

For example, the instruction

add ax, [num1+bx] adds the base address of num1 to the value in bx to determine which memory location to read from.

### 2.4 Branching: Making Decisions

For a program to do anything useful, it needs the ability to make decisions. This is achieved through **comparison** and **conditional branching**.

**Comparison and Conditions**

SUB instruction: **it updates the flags without changing the destination operand**. It performs a subtraction internally to see the relationship between two operands and sets the

**Zero Flag (ZF)**, **Carry Flag (CF)**, **Sign Flag (SF)**, and **Overflow Flag (OF)** accordingly.

# Conditional Jumps

Conditional jump instructions check the state of the flags and transfer program control (i.e., change the Instruction Pointer IP) to a new location if the condition is met. If not, execution simply continues to the next instruction.

Some common conditional jumps include:

- **JNZ (Jump if Not Zero):** Jumps if the Zero Flag is clear (ZF=0). Useful for loops and equality checks.
- **JZ (Jump if Zero) / JE (Jump if Equal):** Jumps if the Zero Flag is set (ZF=1).

**3. Example Codes**

**Example 1: Register Indirect Addressing**

This code adds three numbers stored in memory. It uses BX to hold the address of the current number. Observe how the address in

BX is loaded *without brackets*, but used *with brackets*.

Adding three numbers using indirect addressing

```
[org 0x100]


    mov bx, num1      ; point bx to the address of the first number [cite: 678]
    mov ax, [bx]      ; load the first number (contents of memory at bx) into ax [cite: 679]
    add bx, 2         ; advance bx to point to the second number (words are 2 bytes)
    add ax, [bx]      ; add the second number to ax [cite: 681]
    add bx, 2         ; advance bx to the third number [cite: 682]
    add ax, [bx]      ; add the third number to ax [cite: 683]


    mov ax, 0x4c00    ; terminate program [cite: 686]
    int 0x21
```

```
num1: dw 5, 10, 15, 0
```

**Example 2: Looping with Conditional Jumps**

This example extends the previous concept to add 10 numbers using a loop.

**CX** is used as a loop counter, and **JNZ** checks if the counter has reached zero to terminate the loop.

Adding ten numbers using a loop [cite: 709]

```
[org 0x0100]

    mov bx, num1      ; point bx to the first number [cite: 710]
    mov cx, 10        ; load count of numbers in cx [cite: 711]
    mov ax, 0         ; initialize sum to zero [cite: 712]

l1: add ax, [bx]      ; add the number pointed to by bx [cite: 713]
    add bx, 2         ; advance bx to the next number [cite: 714]
    sub cx, 1         ; decrement the counter [cite: 715]
    jnz l1            ; if cx is not zero, jump back to l1 [cite: 716]

    mov [total], ax   ; store the final sum [cite: 717]

    mov ax, 0x4c00    ; terminate program [cite: 718]
    int 0x21

num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
total: dw 0
```

# Lab Tasks

**Task 1:**

- Store values at DS:1200h and DS:1202h.
- Use BX as a pointer to load these values into registers AX and CX.

**Task 2:**

- Take three numbers and store in memory.
- Use a register as an accumulator.
- Access values from memory indirectly using an offset in the instruction.
- Add them together and save the result in the last memory location.

**Task 3:**

Take three numbers which are stored consecutively in memory (data: db 7, 12, 20, 0).

Write a program to:

- Load each number one by one into registers.
- Add them together.
- Store the result back into the 4th memory location.

**Task 4:**

- Store three word values in memory (dw 7, 12, 20).
- Use a base register as a pointer to access them one by one.
- Add them into **AX**.
- Store the result into a separate memory variable.

**Task 5:**

- Store three word values in memory (dw 7, 12, 20).
- Use **CX** as a counter to repeat the addition process for each value.
- Use **SI** as a pointer to access memory sequentially.

Store the total sum into the variable **result.**